CASE STUDY

Teaching Teachers to Code

Vidcode Professional Development Program

CLIENT
Vidcode

YEAR
2018

TAGS

professional-development · teacher-training · K-12 · curriculum-integration · assessment-design

Between 2017 and 2019, Vidcode developed and delivered a professional development program introducing K-12 teachers to creative coding as a cross-curricular tool. I contributed to the design and facilitation of two formats: a four-day intensive institute (the KP STEM Institute for Creative Coding in any Curriculum, 2018) and a compressed single-day session for the Lyons School District (August 2018). Both were built on the same premise that had shaped the student-facing curriculum: that a teacher with no coding background can run a

creative coding lesson, and that coding belongs in any classroom, not just a computing elective.

The Design Challenge

The teachers in these sessions were not computer science teachers. They were art teachers, history teachers, science teachers — people who had agreed, or been asked, to add coding to their practice without the subject expertise that usually grounds that kind of commitment. The design challenge was not to turn them into programmers in four days. It was to give them enough direct coding experience, enough pedagogical tools, and enough genuine belief in the value of the work, to go back to their classrooms and try.

The frame used throughout was the "teacher/student hat." Teachers were explicitly asked to shift: when coding through a lesson, they put on the student hat. When reviewing a lesson plan or discussing implementation, they put on the teacher hat. This was a practical protocol for moving between two modes of engagement that the PD needed to develop simultaneously: the teacher as a learner of code, and the teacher as a designer of coding experiences for others.

The constraint of the compressed single-day format was its own design problem. The four-day institute could develop understanding gradually. The one-day session had to make the same core case to a whole district, K through 12, on a back-to-school morning, in a single arc. The solution was to identify which elements were non-negotiable — the identity activity, the equity discussion, the assessment calibration protocol — and which could be scaled, and then to differentiate the coding track by grade band: a concurrent CodeSpark webinar for K-3 teachers, while 4-and-above teachers worked through the JavaScript curriculum. Within the 4+ track, branching options allowed more experienced teachers to skip the introductory lessons without the rest of the group waiting.

Program Structure

The four-day institute followed a deliberate daily sequence: What is Vidcode and CS? (Day 1), What does coding look like in the classroom? (Day 2), How can we successfully integrate coding into our curriculum? (Day 3), How do we ensure our students are learning? (Day 4). Each day combined three things: coding as a student, analysis as a teacher, and explicit discussion of the pedagogical choices embedded in the activities.

The coding activities followed a consistent structure: a short background framing, a coding challenge, a structured share (first with a partner, then with the room), a reflection using verbatim prompts, and an extended reflection connecting the technical experience to questions of identity and classroom practice. This was the same gradual release and metacognitive cycle used in the student-facing tutorials, applied to teacher learning. The PD was, in that sense, a demonstration of its own argument.

The one-day session compressed the same structure into six hours: tool tour, individual coding, reflection, paired coding, equity discussion, more coding, debugging, assessment. Each phase built directly on the last. The day closed with the same homework prompt used in the four-day institute: share a finished project with a colleague or on social media. Even in a single day, the goal was to extend the learning into a professional community rather than leave it in the room.

Identity and Equity

Every version of the PD opened with the same activity: groups drew a programmer, shared the drawings, and were then asked whether the drawings reflected the current state of programming or a hoped-for future. Drawings were arranged chronologically around the room. On the final day of the four-day institute, participants returned to those same drawings and reflected on what beliefs had changed.

This arc from assumption to disconfirmation to revised belief was the backbone of the program. The PD was organized around a specific claim: the best way to overcome prejudice and inequity in computer science is to integrate it into the school day. The programmer drawing activity was how that claim was made experiential rather than simply stated. By Day 4, participants who had drawn a solitary white male at a terminal four days earlier were holding the drawing next to the projects they had made and the students they had thought about, and the gap between the drawing and the reality was the argument.

The equity thread ran through the coding activities as well. The meme challenge asked teachers to code an anti-stereotype meme. An example PSA project used content from the Black Lives Matter movement. A data visualization project used sea ice extent data from NSIDC. The facilitation guide prompted explicit discussion of gendered language in the classroom, inclusive project prompts, and what differentiation looks like in a creative coding context. The technical and the human were kept aligned throughout, as they were in the student curriculum.

Assessment Literacy

The rubric used in both programs assessed five dimensions: Video Content, Code Execution, Code Practice, Reflection, and Habits of Mind. The Habits of Mind row was a deliberate design choice. It assessed dispositions rather than performance. At the Distinguished level, a student "embraces the goal of the program and chooses to try out new ideas and multiple solutions, even when they are challenging." At the Unsatisfactory level, a student "is not aware of the goal of the program, is frequently off-task, does not offer their own ideas, and gives up when it is difficult." Making this dimension explicit meant that teachers were grading something that most coding rubrics ignore: the quality of engagement and persistence that determines whether a student will actually become a learner of code.

Rather than presenting the rubric and discussing it abstractly, both programs used a calibration protocol. Teachers annotated the rubric dimensions, then applied the rubric to real student exemplar projects (live URLs from the platform, tagged by course level and production notes), then cross-assessed a peer's work, then found what the agenda called the "stickiest sample" — a project where group members disagreed — and worked toward consensus. This is a standard teacher assessment literacy approach, but it is rarely used in PD for creative or non-tested subjects. Applying it to creative coding projects made a serious claim: this work is assessable, and developing the judgment to assess it consistently is a professional skill worth practicing.

The debugging exercises used the same taxonomic approach. Three tiers of broken JavaScript code moved from single syntax errors to multiple errors to logic errors that required understanding program behavior, not just reading for typos. Error types were labeled explicitly: KEYWORD, VARIABLE NAMES, STRING, ARGUMENTS, LOGIC, REFERENCE ERROR, SEQUENCE, PAIRED BRACKETS, UNEXPECTED BEHAVIOUR. By naming error types, the exercises built a diagnostic vocabulary rather than just a repair habit.

Reflection

The teacher/student hat framing worked because it named something that professional development often leaves implicit: the people in the room are being asked to learn and to teach simultaneously, and those are different modes that require different kinds of attention. Making the shift explicit reduced the anxiety that often comes with coding publicly in front of colleagues, and redirected it toward the pedagogical analysis that was actually the point.

What I would revisit is the single-day format's relationship to follow-through. The compressed session produced genuine engagement and, by the end, real confidence in most participants. But the evidence base for single-day PD producing sustained classroom change is thin, and the design acknowledged this only implicitly. A more structured follow-up model — asynchronous check-ins over the following weeks, a shared project gallery, even a light accountability structure — would have made the day more likely to translate. The homework prompt to share on social media was a start, but it was also the place where the design ran out of room.
